S7 DELIVERY CONTROL CENTRE

Launch Walkthrough

How to set up, launch and navigate the S7 Delivery Control Centre — the customer-safe console that runs the governed AI-assisted delivery pipeline end to end, offline, with no API key required.

1 Before you start

You need **Python 3.12 or later** and a terminal. Nothing else — no API key, no external service, no internet connection once the repository is cloned. The Control Centre runs entirely on deterministic simulation, so every run is reproducible and every artifact is honestly labelled `SIMULATED` or `HUMAN`.

> **No API key needed.** The Control Centre never calls a live model. Everything you see is generated by the demo engine — clearly badged as such throughout the interface.

2 Set up the environment

From the repository root:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

This installs the pinned dependencies (FastAPI, Uvicorn, Pydantic and a handful of others) into an isolated virtual environment. It takes under a minute on a typical connection.

3 Launch the console

```
demo/run_control.sh
```

This starts a local web server bound to `127.0.0.1` — a presenter's local demo, not a public service. By default it listens on **port 8720**. Once it prints `Uvicorn running on http://127.0.0.1:8720`, open that address in a browser.

Custom port

If 8720 is already in use, set `PORT` before launching:

```
PORT=8730 demo/run_control.sh
```

Stopping the server

Press `Ctrl-C` in the terminal running the script. All run state lives on disk under `artifacts/runs/` — stopping and restarting the server loses nothing.

4 Find your way around

The console has three fixed regions and one changing one:

REGION	WHAT IT'S FOR
Top bar	Scenario, Run ID, Environment (Demo / Replay), the "Customer-safe view" badge, and the **role switcher** — who you're acting as changes which actions you're allowed to take.
Stepper	The five delivery stages — Intake, Planning, Build & Review, Quality, Release. Click any stage to jump to it.
Left sidebar	Section navigation within a stage (e.g. under Planning: Epic to Stories, Dependency Map, Routing by Team…), plus governance views (Traceability, Provenance, Activity Log, Risks).
Main panel	Renders entirely from `GET /api/runs/<id>` — nothing here is invented client-side.

Runs

A **run** is one pass through the pipeline for a scenario. The Run ID selector in the top bar switches between existing runs, or pick **Reset demo** (footer, bottom right) to restore the seeded starting state at any time.

Roles

Every action is permission-checked server-side against the acting role. Switch roles from the top-right selector to see what each can and can't do:

ROLE	CAN TYPICALLY
`delivery_lead`	Drive intake and planning actions; the default role.
`business_owner`	Sign off the plan at Gate 1.
`engineering_lead`	Start/develop tasks; also unlocks the **Technical Detail** view on Development Progress (step-level process, skills used, files changed — outside the customer-safe guarantee for other roles).
`qa_lead`	Run quality checks and decide the quality gate; same Technical Detail access as engineering lead.
`independent_reviewer`	Execute the independent review — isolated from development, cannot approve its own output.
`release_manager`	The only role that can deploy.

5 A guided first pass

To see the whole pipeline in a few minutes, pick a run that's already complete from the Run ID dropdown (an early one such as `S7-00002` typically is), and walk the stepper left to right:

1. **Intake** — Requirement Summary is collapsed by default; expand it for the full requirement. Below it, AI Analysis shows the extracted business rules and clarification questions. Intake Gate sits at the bottom, also expandable, and shows which conditions passed.
2. **Planning → Epic to Stories** — the decomposed user stories. Click any row for the full story detail (purpose, acceptance criteria, feature flag, rollback plan).
3. **Planning → Dependency Map** — the same stories as a dependency graph, defaulting to top-to-bottom.

4. **Build & Review → Work Queue** — every task, its team and status. Click a row to focus it; the right-hand rail fills in with owner, dependencies and last activity.
5. **Build & Review → Independent Review** — the governance centrepiece. A task that was blocked and later corrected shows *both* review versions: the original major finding and the passing re-review, so the "no phase self-approves" story stays visible even after the fix landed.
6. **Quality** and **Release** — explicit gate conditions, never a score; four named approvals before deploy.

> **Look for the badges.** Every artifact carries a provenance label — `SIMULATED` (produced by the deterministic demo engine) or `HUMAN` (a person's own input). Nothing simulated is ever presented as if it were live.

---

6 The other two surfaces

The Control Centre is one of three surfaces over the same underlying pipeline. You won't need the other two for a Control Centre walkthrough, but it's worth knowing they exist:

SURFACE	LAUNCH	PORT	WHAT
Control Centre	`demo/run_control.sh`	8720	APP The end-to-end governed run, described above.
Delivery console	`demo/run_console.sh`	8700	APP The original five-gate upstream console.
Intake	`demo/run_intake.sh`	8710	APP Epic intake, clarify → plan.

---

7 Troubleshooting

"address already in use"

Another process is already bound to the port. Either stop it, or launch with a different `PORT` as shown above.

A gate button is greyed out

You're likely acting as a role that isn't permitted for that action — check the role selector in the top bar. The engine enforces this server-side; the UI reflects it but never invents the restriction.

Something looks stuck after a refresh

Click the refresh icon (⟳) next to the clock in the top bar to re-fetch run state, or switch to a different run and back.